epsi
l'école d'ingénierie informatique

COMPTE RENDU DE PROJET

PROJET TRANSVERSAL — ENVIRONNEMENT WEB



Clavardage

Application de messagerie instantanée — PHP / MVC / MySQL

WIEREZ Mylo • FREIHUBER Théo • GRAUET Mathis • ANDRIES Alexis

B1 – 2025-26

Rendu le : 26/01/2026

Sommaire

Présentation 3
- Thème du projet 3
- Objectifs 3

Analyse du sujet 3
- Description fonctionnelle 3
- Contraintes techniques 4

Méthodologie 4
- Approche de développement 4
- Répartition du travail 4
- Environnement et outils 5

Réalisation 5
- Environnement — Machine Virtuelle 5
- Architecture du projet (MVC) 6
- Base de données 6
- Backend — Contrôleurs PHP 7
- Frontend — Vues et interface 9
- Actualisation automatique (polling) 10
- Sécurité 10

Synthèse 11
- Bilan général 11
- Difficultés rencontrées 11
- Limites identifiées 11
- Améliorations possibles 12

Conclusion 12

1. PRÉSENTATION

Thème du projet

Ce projet transversal, réalisé dans le cadre du cours **Environnement Web** de première année de Bachelor, consiste en la conception et le développement d'une **application de clavardage** (messagerie instantanée en ligne). L'objectif est de produire une application web complète et fonctionnelle permettant à des utilisateurs inscrits d'échanger des messages en temps quasi réel, les données étant persistées dans une base de données MySQL.

Le terme « clavardage » (contraction de clavier et bavardage) désigne la communication écrite instantanée sur Internet. L'application porte le nom de marque

Clavardon et propose un environnement de chat privé, en tête-à-tête, avec gestion complète des comptes utilisateurs.

Objectifs

Les objectifs du projet couvrent deux dimensions complémentaires :

- **Objectif fonctionnel :** produire une application web opérationnelle permettant l'inscription, la connexion, la création de conversations privées, l'envoi et la réception de messages persistés en base de données, la modification et la suppression de profil.
- **Objectif technique :** maîtriser une architecture MVC (Modèle-Vue-Contrôleur) en PHP natif, manipuler une base de données relationnelle via PDO, sécuriser l'accès aux pages protégées, et assurer la cohérence des données entre les tables lors des opérations d'écriture.
- **Objectif pédagogique :** intégrer les langages HTML, CSS, JavaScript, PHP et SQL dans un projet cohérent et réaliste, en reproduisant les mécanismes des applications web professionnelles (routage, sessions, requêtes préparées, guard de sécurité).

2. ANALYSE DU SUJET

Description fonctionnelle

L'application Clavardon devait répondre aux exigences fonctionnelles suivantes :

- **Gestion des utilisateurs :** inscription (login, email, mot de passe), connexion, déconnexion, modification des informations du profil, suppression du compte.
- **Gestion des conversations :** création d'une nouvelle conversation privée avec un autre utilisateur, liste des conversations existantes avec aperçu du dernier message, suppression d'une conversation (avec ses messages associés).
- **Messagerie :** envoi d'un message dans une conversation active, affichage de l'historique des messages avec la date d'envoi, actualisation automatique pour afficher les nouveaux messages.
- **Rôle administrateur :** un utilisateur avec le flag admin = 1 accède à la liste complète de toutes les conversations de la plateforme et peut supprimer n'importe quelle discussion.

- **Persistance :** l'intégralité des données (utilisateurs, discussions, messages) est stockée en base de données MySQL, de sorte que l'utilisateur retrouve son historique complet à chaque reconnexion.

Contraintes techniques

- **Langage serveur :** PHP natif (sans framework), avec PDO pour les accès base de données.
- **Architecture :** séparation contrôleurs / vues imposée, selon un pattern MVC allégé.
- **Routage :** un fichier .htaccess redirige toutes les URL vers index.php pour centraliser le point d'entrée de l'application.
- **Environnement de déploiement :** serveur Apache hébergé sur une machine virtuelle Linux configurée spécifiquement pour le projet.
- **Sécurité :** toutes les pages sensibles doivent être protégées par une vérification de session ; les requêtes SQL doivent utiliser des requêtes préparées (PDO) pour prévenir les injections.
- **Interface utilisateur :** HTML/CSS responsive, JavaScript pour l'actualisation automatique des messages.

3. MÉTHODOLOGIE

Approche de développement

Le développement a suivi une approche par couches successives, du bas vers le haut :

- **Phase 1 — Infrastructure :** configuration de la machine virtuelle, installation d'Apache, PHP et MySQL, mise en place du fichier .htaccess.
- **Phase 2 — Modélisation :** conception du schéma de base de données (tables user, discussion, chat) et définition des relations entre elles.
- **Phase 3 — Arborescence :** mise en place de la structure de dossiers (controller/ et vue/) pour séparer logique métier et affichage.
- **Phase 4 — Frontend :** intégration des maquettes HTML/CSS pour l'ensemble des pages (connexion, inscription, conversations, chat, profil).
- **Phase 5 — Backend :** implémentation de chaque contrôleur PHP (login, signin, logout, createchat, message, modifprofil, delprofil, deletchat) avec connexion à la base de données via PDO.
- **Phase 6 — Tests et intégration :** validation du bon fonctionnement des flux complets (inscription → connexion → chat → déconnexion) et correction des anomalies.

Répartition du travail

Le projet a été réalisé en groupe de quatre. Chaque membre s'est vu confier un périmètre fonctionnel cohérent :

Membre	Domaine principal	Fichiers associés
ANDRIES Alexis	Backend messagerie &	createchat.php, message.php,

Membre	Domaine principal	Fichiers associés
	conversations	deletchat.php, chat.php
FREIHUBER Théo	Infrastructure & routage	Configuration VM, .htaccess, index.php, welcome.php
GRAUET Mathis	Authentification & profil	login.php, signin.php, logout.php, modifprofil.php, delprofil.php
WIEREZ Mylo	Frontend & CSS	style.css, conversations.php, nouvelle_conversation.php, profil.php, signin.php, login.php

Des sessions d'intégration régulières ont permis d'aligner les interfaces entre contrôleurs et vues, de mettre à jour les sessions PHP au fil des opérations, et de valider la cohérence des données entre les trois tables de la base.

Environnement et outils

- **Langages :** PHP 8.x (serveur), HTML5/CSS3 (présentation), JavaScript (actualisation), SQL (base de données).
- **Serveur :** Apache sur machine virtuelle Linux (Ubuntu Server).
- **Base de données :** MySQL, base clavarde, accès via PDO.
- **Éditeurs :** VS Code avec extensions PHP Intelephense et Live Server, PHPStorm.
- **Versioning :** partage de code et synchronisation des fichiers entre membres.
- **Documentation :** php.net (PDO, sessions, fonctions array), MDN Web Docs (HTML/CSS/JS).

4. RÉALISATION

Environnement — Machine Virtuelle

La première étape du projet a consisté à créer et configurer la machine virtuelle qui héberge l'application. Cette VM joue le rôle de serveur de développement local, reproduisant fidèlement les conditions d'un hébergement web réel.

La configuration comprend l'installation d'Apache (serveur HTTP), de PHP et du serveur MySQL, ainsi que la mise en place des droits d'accès et des modules nécessaires (mod_rewrite pour le fichier .htaccess).

```
dnf -y update
dnf -y install php
dnf -y install epel-release
dnf -y install mariadb-server mariadb
dnf -y install httpd
dnf -y install phpmyadmin
ss -atnp | grep LISTEN
systemctl status httpd
systemctl enable httpd
systemctl status httpd
ss -atnp | grep LISTEN
systemctl start php
ss -atnp | grep LISTEN
curl 127.0.0.1
```

Figure 1 — Configuration de la machine virtuelle

Architecture du projet (MVC)

Le projet adopte une architecture inspirée du patron MVC (Modèle-Vue-Contrôleur), adaptée aux contraintes du PHP natif :

- **index.php :** point d'entrée unique de l'application, inclut la vue d'accueil (welcome.php). Toutes les URL non résolues sont redirigées ici par le fichier .htaccess.
- **controller/ :** contient la logique métier — interaction avec la base de données, traitement des formulaires, gestion des sessions, redirections. Chaque contrôleur est spécialisé dans une fonctionnalité.
- **vue/ :** contient les fichiers PHP/HTML de présentation. Chaque vue requiert guard.php pour vérifier que l'utilisateur est authentifié, puis affiche les données issues du contrôleur correspondant.

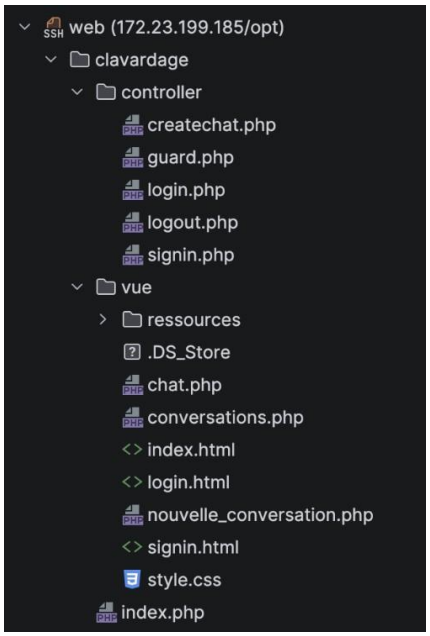


Figure 2 — Arborescence du projet

Le fichier .htaccess active la réécriture d'URL et redirige toutes les requêtes non résolues vers index.php, centralisant ainsi le point d'entrée et simplifiant la gestion des routes :

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_DIRNAME} !-d
RewriteRule ^.+ /index.php [R=301,L]
```

Base de données

Schéma relationnel

La base de données clavarde est composée de trois tables dont les relations assurent la cohérence de l'ensemble des données :

Table	Colonnes principales	Rôle
user	user_id (PK), login, email, password, admin	Stocke les comptes utilisateurs. Le champ admin (0/1) détermine les droits étendus.
discussion	id_discussion (PK), membre1, membre2	Représente une conversation privée entre deux utilisateurs (logins).
chat	id_message (PK), envoyeur, message, date, id_discussion (FK)	Stocke chaque message envoyé, lié à une discussion.

Cohérence des données en cascade

Un soin particulier a été apporté à la cohérence des données lors des opérations de modification et de suppression. Par exemple, lorsqu'un utilisateur modifie son login dans modifprofil.php, trois tables sont mises à jour en cascade :

```
-- Mise à jour du login dans la table user
UPDATE user SET login = :login, email = :email, password = :password WHERE user_id
= :user_id;

-- Propagation dans la table discussion
UPDATE discussion SET
  membre1 = CASE WHEN membre1 = :oldlogin THEN :newlogin ELSE membre1 END,
  membre2 = CASE WHEN membre2 = :oldlogin THEN :newlogin ELSE membre2 END
WHERE membre1 = :oldlogin OR membre2 = :oldlogin;

-- Propagation dans la table chat
UPDATE chat SET envoyeur = :newlogin WHERE envoyeur = :oldlogin;
```

Ce mécanisme garantit qu'un changement de login ne brise pas les références dans les tables discussion et chat, maintenant ainsi l'intégrité référentielle de la base.

Backend — Contrôleurs PHP

Organisation générale

Chaque contrôleur suit le même patron : inclure guard.php pour vérifier la session, récupérer les données POST/GET/SESSION, appeler la ou les fonctions métier correspondantes, puis rediriger

l'utilisateur vers la vue appropriée. La connexion PDO est systématiquement encapsulée dans une fonction getDb() locale :

```
function getDb(): PDO {
    $db = new PDO('mysql:host=localhost;dbname=clavarde;charset=utf8', 'root',
'mdp');
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    return $db;
}
```

Le tableau ci-dessous résume le rôle de chacun des 9 contrôleurs :

Contrôleur	Rôle
guard.php	Vérifie la présence du cookie de session et du flag $_SESSION['is_logged']. Redirige vers index.php si la vérification échoue.
login.php	Authentifie l'utilisateur (login + mot de passe). En cas de succès, initialise la session et redirige vers conversations.php.
signin.php	Inscrit un nouvel utilisateur. Vérifie l'unicité du login avant l'insertion en base.
logout.php	Détruit la session ($_SESSION = array(); session_destroy()) et redirige vers index.php.
createchat.php	Crée une nouvelle conversation après avoir vérifié l'existence du destinataire et l'absence de doublon de discussion.
message.php	Insère un nouveau message (envoyeur, contenu, date, id_discussion) dans la table chat.
deletchat.php	Supprime une conversation (et ses messages) par id_discussion.
modifprofil.php	Met à jour login, email et mot de passe dans user, discussion et chat en cascade.
delprofil.php	Supprime le compte utilisateur, détruit la session et redirige vers index.php.

Contrôleur login.php — Authentification

L'authentification compare le login et le mot de passe saisis avec les enregistrements en base. En cas de correspondance, les informations essentielles (user_id, username, email, admin) sont stockées en session pour être accessibles sur toutes les pages protégées :

```
$stmt->execute(['login' => $login, 'password' => $password]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);
if ($user) {
    session_start();
    $_SESSION['is_logged'] = true;
    $_SESSION['user_id']   = $user['user_id'];
    $_SESSION['username']  = $user['login'];
    $_SESSION['admin']     = $user['admin'];
    header('Location: /vue/conversations.php');
}
```

Contrôleur createchat.php — Création de conversation

Avant d'insérer une nouvelle discussion, le contrôleur effectue deux vérifications : l'existence du destinataire dans la table user, et l'absence d'une conversation déjà existante entre les deux membres (dans les deux sens) :

```
foreach ($allConversation as $conv) {
    if (($conv['membre1'] == $login && $conv['membre2'] == $destinataire) ||
        ($conv['membre1'] == $destinataire && $conv['membre2'] == $login)) {
        $conversationExiste = true; break;
    }
}
```

Il est également impossible pour un utilisateur d'ouvrir une conversation avec lui-même (vérification $destinataire === $login).

Frontend — Vues et interface

Pages de l'application

L'interface utilisateur est constituée de 8 pages PHP/HTML, chacune incluant guard.php pour protéger son accès :

Vue	Description
welcome.php	Page d'accueil publique présentant l'application avec liens vers connexion et inscription.
login.php	Formulaire de connexion (login + mot de passe). Soumet vers controller/login.php.
signin.php	Formulaire d'inscription (login, email, mot de passe). Soumet vers controller/signin.php.
conversations.php	Liste les conversations de l'utilisateur connecté avec aperçu du dernier message et bouton de suppression. Vue admin étendue.
chat.php	Affiche l'historique des messages d'une conversation et le formulaire d'envoi. Inclut attendre.js pour l'actualisation.
nouvelle_conversation.php	Formulaire de création d'une nouvelle discussion (saisie du login du destinataire).
profil.php	Affiche les informations du compte et propose la modification ou la suppression du profil.

Design et CSS

L'interface Clavardon adopte un design moderne et épuré, entièrement codé en CSS natif (style.css). Les choix graphiques comprennent une palette de couleurs cohérente, des composants réutilisables (banners, boutons, cards de conversation), un layout responsive adapté aux différentes tailles d'écran, et des transitions CSS pour les interactions utilisateur.

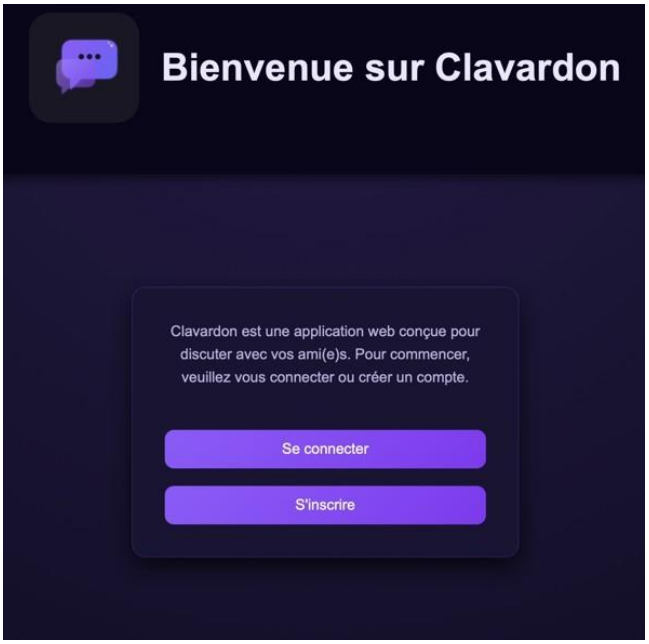


Figure 3 — Aperçu de l'interface utilisateur (page Conversations)

Actualisation automatique (polling)

Pour simuler la réception de messages en quasi temps réel, le fichier attendre.js implémente une technique de polling côté client : la page de chat se recharge automatiquement toutes les 15 secondes.

```
setTimeout(function() { location.reload(); }, 15000);
```

Ce fichier est inclus dans chat.php et conversations.php via une balise *<script src="attendre.js"></script>*. Cette approche simple garantit la fraîcheur des messages sans nécessiter de WebSocket ni d'API dédiée, au prix d'un rechargement complet de la page toutes les 15 secondes.

Sécurité

Guard de session — guard.php

Toutes les pages protégées (conversations, chat, profil) débutent par require '../controller/guard.php'. Ce fichier vérifie la présence du cookie de session PHPSESSID et du flag $_SESSION['is_logged']. Si l'une des deux conditions n'est pas remplie, l'utilisateur est immédiatement redirigé vers la page d'accueil :

```
if (isset($_COOKIE['PHPSESSID'])) { session_start(); }
if (!isset($_SESSION['is_logged']) || !$_SESSION['is_logged']) {
    header('Location: ../index.php');
    exit;
}
```

Ce mécanisme centralise le contrôle d'accès et évite toute répétition de la vérification dans chaque fichier vue.

Requêtes préparées PDO

L'ensemble des requêtes SQL utilise des requêtes préparées avec bindValue() ou des tableaux de paramètres nommés. Cette pratique protège l'application contre les injections SQL en séparant strictement le code SQL des données utilisateur :

```
$sql  = 'SELECT * FROM user WHERE login = :login';
$stmt = $pdo->prepare($sql);
$stmt->execute(['login' => $destinataire]);
```

Les entiers (identifiants) sont typés explicitement via PDO::PARAM_INT pour empêcher toute conversion de type inattendue.

Échappement des sorties HTML

Toutes les données dynamiques affichées dans les vues sont passées par htmlspecialchars() avant l'affichage, ce qui prévient les attaques de type Cross-Site Scripting (XSS) :

```
echo htmlspecialchars($conversation['username']);
echo htmlspecialchars($conversation['last_message']);
```

5. SYNTHÈSE

Bilan général

Le projet Clavardon remplit ses objectifs principaux : l'application est fonctionnelle, déployée sur la machine virtuelle, et couvre l'ensemble des fonctionnalités demandées — inscription, connexion, messagerie privée persistée, gestion du profil et rôle administrateur.

Du point de vue pédagogique, ce projet a constitué une mise en pratique complète et cohérente de l'ensemble du stack web étudié en cours : HTML/CSS pour la présentation, PHP natif pour la logique serveur, PDO/MySQL pour la persistance, et JavaScript pour l'interactivité côté client. L'architecture MVC adoptée a imposé une rigueur de séparation des responsabilités qui a facilité le travail en équipe.

La collaboration à quatre a été enrichissante : la répartition claire des périmètres a permis de travailler en parallèle, et les sessions d'intégration ont développé des réflexes de communication technique essentiels (conventions de nommage, interfaces entre modules, gestion des variables de session).

Difficultés rencontrées

- **Cohérence des sessions PHP :** assurer que les variables de session ($login, $user_id, $admin, $id_discussion, $url) soient correctement initialisées, mises à jour et transmises entre les contrôleurs et les vues a représenté la principale source de bugs lors de l'intégration.
- **Cascade de mise à jour :** implémenter correctement la mise à jour en cascade lors du changement de login (tables user, discussion et chat) a nécessité une analyse fine des dépendances entre tables.

- **Routage et redirections :** configurer correctement le fichier .htaccess et gérer les chemins relatifs (../vue/ vs /vue/) dans les redirections header() s'est révélé délicat en raison des différences entre les chemins filesystem et les URL web.
- **Configuration de la VM :** la mise en place de l'environnement serveur (activation de mod_rewrite, droits Apache, configuration MySQL) a pris un temps significatif en début de projet.

Limites identifiées

- **Mots de passe en clair :** les mots de passe sont stockés en clair dans la base de données. L'utilisation de password_hash() et password_verify() est impérative pour tout passage en production.
- **Polling vs WebSocket :** le rechargement toutes les 15 secondes est acceptable en développement mais génère une charge serveur inutile et une latence d'affichage non négligeable. Une implémentation via WebSocket ou Server-Sent Events serait plus performante.
- **Conversations uniquement en 1:1 :** l'application ne supporte que les discussions à deux participants. Les groupes de conversation ne sont pas implémentés.
- **Absence de pagination :** l'intégralité de l'historique des messages est chargée à chaque ouverture d'une conversation, ce qui peut devenir problématique pour des échanges de longue durée.
- **Validation des entrées :** certains contrôleurs manquent de validation fine des données saisies (longueur maximale, caractères autorisés), laissant la porte ouverte à des comportements inattendus.

Améliorations possibles

- **Hachage des mots de passe :** adopter password_hash()/password_verify() pour ne jamais stocker de mot de passe en clair.
- **Actualisation temps réel :** remplacer le polling JavaScript par des WebSockets (Ratchet/PHP) ou des Server-Sent Events pour une expérience utilisateur fluide et une consommation serveur réduite.
- **Conversations de groupe :** adapter le schéma de base de données pour supporter N membres par discussion (table discussion_membre).
- **Pagination des messages :** charger l'historique par tranches (LIMIT/OFFSET) et implémenter un défilement infini vers le haut pour les conversations longues.
- **Notifications :** afficher un badge de messages non lus dans la liste des conversations pour indiquer à l'utilisateur qu'il a des nouveaux messages.
- **Upload de médias :** permettre l'envoi de photos et de fichiers dans les conversations.
- **Mode responsive amélioré :** optimiser l'interface pour les appareils mobiles avec un menu hamburger et un layout en colonne unique.

6. CONCLUSION

Le projet Clavardon représente une synthèse concrète et ambitieuse des compétences acquises en première année de Bachelor dans le domaine du développement web. En partant de zéro — depuis la configuration de la machine virtuelle jusqu'au déploiement d'une application multi-utilisateurs

complète — le groupe a couvert l'ensemble du cycle de développement d'une application web moderne.

L'adoption d'une architecture MVC, même en PHP natif, a démontré les bénéfices d'une séparation rigoureuse des responsabilités : meilleure lisibilité du code, répartition efficace des tâches entre membres de l'équipe, et facilité de maintenance et d'évolution. L'utilisation systématique de PDO avec requêtes préparées et d'htmlspecialchars() témoigne d'une prise de conscience des enjeux de sécurité web.

Les limites identifiées — notamment le stockage des mots de passe en clair et le mécanisme de polling — dessinent une feuille de route claire pour porter Clavardon au niveau d'une application de production : hachage des mots de passe, WebSockets, pagination, notifications. Ces points d'amélioration seront autant d'opportunités d'approfondissement dans les prochains cours et projets.

Ce projet a également renforcé des compétences transversales essentielles : communication technique au sein d'une équipe de quatre, gestion des dépendances entre modules, intégration de contributions parallèles et résolution collaborative de problèmes. Ces compétences s'avéreront aussi précieuses que les compétences techniques dans les environnements professionnels futurs.